

Design and Analysis of Algorithm

Lecture 23: Queue

Content



- **Priority Queue**

Priority Queue

In Priority Queue the order in which elements are deleted and processed comes from the following rules:

An element of higher priority is processed before any element of lower priority.

Two elements with the same priority are processed according to the order in which they were added to the queue.

Priority Queue: One Way Representation

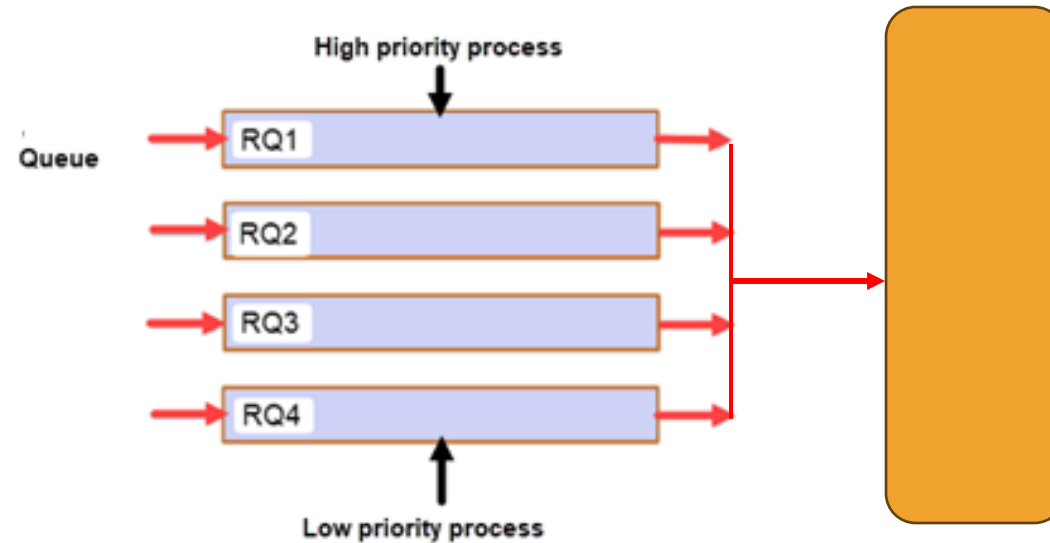
One way to maintain a priority queue in memory is by means of a one-way list, as follows:

- Each node in the list will contain three items of information: an information field INFO, a priority number PRN and a link number LINK.
- A node X precedes a node Y in the list
 - (1) when X has higher priority than Y or
 - (2) when both have the same priority but X was added to the list before Y. This means that the order in the one-

Priority Queue: Array Representation

Another way to maintain a priority queue in memory is to use a separate queue for each level of priority

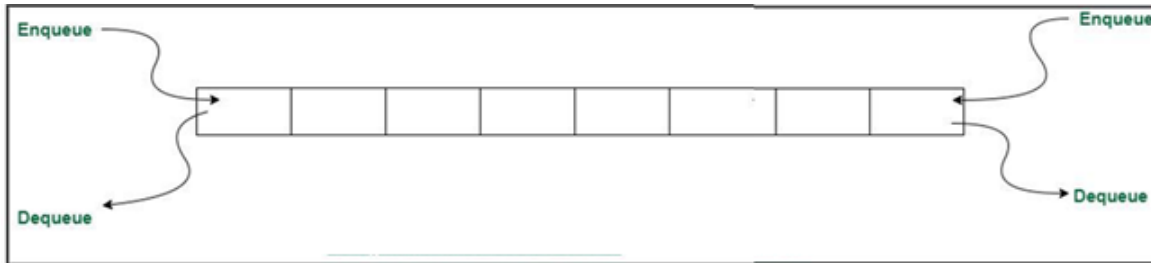
- Each such queue will appear in its own circular array and must have its own pair of pointers, FRONT and REAR.



Double Ended Queue

A deque (double-ended queue) allows insertions and deletions at both front and rear .

Example: Rotational Operations



Applications requiring to perform clock-wise and anti-clockwise rotational operations.

Double Ended Queue: Array Implementation

Deletion from FRONT

- Increase FRONT

$(\text{FRONT} + 1) \% N$

Insertion at FRONT

- Decrease FRONT

$(\text{FRONT} - 1 + N) \% N$

Insertion at REAR

- Increase REAR

$(\text{REAR} + 1) \% N$

Deletion from REAR

- Decrease REAR

$(\text{REAR} - 1 + N) \% N$

Question 1

A queue is implemented using an array such that *ENQUEUE* and *DEQUEUE* operations are performed efficiently. Which one of the following statements is CORRECT

1. Time Complexity of Enqueue and Dequeue operations are $O(n)$
2. Time Complexity of Enqueue is $O(1)$ but for Dequeue operations is $O(n)$
3. Time Complexity of Enqueue is $O(n)$ but for Dequeue operations is $O(1)$
4. Time Complexity of Enqueue and Dequeue operations are $O(1)$

Question 2

Consider a sequence a of elements $a_0 = 1, a_1 = 5, a_2 = 7, a_3 = 8, a_4 = 9$, and $a_5 = 2$. The following operations are performed on a stack S and a queue Q , both of which are initially empty.

8
7
8
7
5
1

Stack

Queue

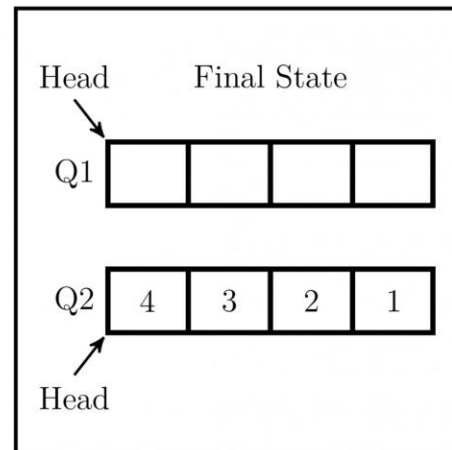
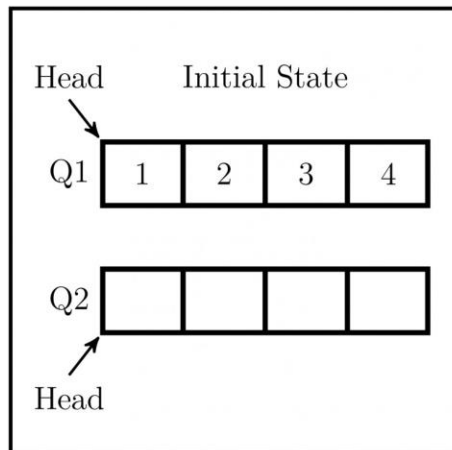
1. Push the elements of a from a_0 to a_5 in that order into S .
2. Enqueue the elements of a from a_0 to a_5 in that order into Q .
3. Pop an element from S .
4. Dequeue an element from Q .
5. Pop an element from S .
6. Dequeue an element from Q .
7. Dequeue an element from Q and push the same element into S .
8. Repeat Last operation three times.
9. Pop an element from S .
10. Pop an element from S .

The top element of after executing the above operations is

_____.

Question 3

Consider the queues Q_1 containing four elements and Q_2 containing none (shown as the in the figure). The only operations allowed on these two queues are Enqueue (Q, Element) and Dequeue (Q). The minimum number of Enqueue operations on Q_1 required to place the elements of Q_1 in reverse order (shown as the in the figure) without using any additional storage is _____.



Step 0: Enqueue 1 in Q2, it is already sorted,

Step 1: Enqueue 2 in Q2 i.e 1,2

Step 2: perform 1 dequeue and 1 enqueue simultaneously for 1 time in Q2 i.e 2,1 (Head is pointing at 2)

Step 3: enqueue 3 in Q2 i.e 2,1,3

Step 4: perform 1 dequeue and 1 enqueues in simultaneously for 2 times in Q2 i.e 2,1,3 → 1,3,2 → 3,2,1 (head pointing at 3 now)

Step 5: Enqueue 4 in Q2 i.e 3,2,1,4

Step 6: perform 1 dequeue and 1 enqueues in simultaneously for 3 times in Q2 i.e 3,2,1,4 → 2,1,4,3 → 1,4,3,2 → 4,3,2,1

So the number of enqueue operations in Q1 is 0.

Question 4

What operation does the function `process(Q)` perform on the queue?

```
void process(queue *Q) {  
    int num;  
    if (!is_queue_empty(Q)) {  
        num = dequeue(Q);  
        process(Q);  
        enqueue(Q, num);  
    }  
}
```

1. Reverses the Queue
2. Deletes the front element and add it to the rear without modifying anything else
3. Empties the Queue
4. No Change in the Queue

where,

- `is_queue_empty(Q)`** – Returns 1 if the queue is empty, otherwise 0
- `dequeue(Q)`** – Removes and returns the front element of the queue.
- `enqueue(Q, val)`** – Inserts integer *val* at the rear of the queue.